

Assignment 4: E-Commerce Database Application Report

Roger Arias & Irene Ramirez

Date Submitted: 12/2/2025

Course: COP 3703 Introduction to Databases

I. Application & Design Overview

A. Technology Stack

- **GUI & Core Application:** Python with Tkinter (using ttk for styling).
- **Database Connectivity:** PostgreSQL, using the psycopg2 library.
- **Architecture:** Two main Python files: app.py (handles the GUI/UI logic) and db_manager.py (handles all database interaction).

B. Group Contribution Breakdown

Member Name	Specific Contribution
-------------	-----------------------

Roger Arias	
-------------	--

- | | |
|--|---|
| | <ul style="list-style-type: none">- Implemented the entire DBManager class (db_manager.py) for database connection and query execution.- Wrote the Python logic for the Multi-Step Transaction (Atomicity) including rollback/commit handling.- Developed and executed the Unit Tests (unittest.py) to verify all business logic.- Designed and created the PostgreSQL Stored Functions (place_new_order, finalize_order_total). |
|--|---|

Irene Ramirez	
---------------	--

- | | |
|--|---|
| | <ul style="list-style-type: none">- Designed and implemented the complete GUI structure using Tkinter (app.py).- Developed the Login System and role verification UI (Admin/Customer). |
|--|---|

- Implemented the Admin Dashboard features, including displaying Restock Alerts.

- Implemented the Customer Dashboard for placing and viewing orders.

II. Database Interaction and Business Logic

A. Database Connection & Setup

- **Credentials:** The application connects using fixed parameters (localhost, 5432, postgres, Assignment2) defined in db_manager.py.
- **Connection Handling:** The connection is initialized upon application start and is managed using a single DBManager instance passed throughout the app. A fatal error message is displayed if the connection fails.

B. Implementation of Stored Functions (Business Logic)

Design Decision: We chose to implement the critical order creation and finalization logic as stored functions in PostgreSQL to ensure the logic resides in the database layer, enforcing data integrity and consistency regardless of the client application.

1. **place_new_order(customer_id, shipping_address, payment_method)**

- **Purpose:** Inserts a new row into the Orders table and returns the new orderID.
- **Implementation Note:** This stored function handles the initial order creation step before any items are added.

2. **finalize_order_total(order_id)**

- **Purpose:** Recalculates the total of an existing order by summing the subtotal field of all associated OrderItems, and updates the totalAmount field in the Orders table.
- **Implementation Note:** This ensures the final total is accurate and based on committed data.

C. Multi-Step Transaction (Atomicity)

We implemented the transaction logic to ensure the order process is atomic. This is critical for maintaining data integrity, especially stock levels and order totals.

1. **Transaction Start:** The application first calls the `place_new_order` stored function to generate a unique `orderId`.
2. **Per-Item Transaction (`add_order_item_and_update_stock`):**
 - **Stock Check:** The function acquires a lock, verifies available stock against the requested quantity.
 - **Action:** If stock is sufficient, the item is inserted into `OrderItems` and `stockQuantity` is decremented in `Products`.
 - **Commit/Rollback:** If all updates succeed, the transaction is committed (COMMIT). If the stock check fails, or any other DB error occurs, the entire transaction for that item is rolled back (ROLLBACK), preventing partial updates and stock corruption.
3. **Transaction Finish:** The application calls `finalize_order_total` to calculate and save the final total for the entire order.

III. Verification and Edge Cases

A. Admin Feature: Restock Alerts

- **Method:** `DBManager.get_restock_alerts(threshold)`
- **Functionality:** Correctly queries the `Products` table for all items where `stockQuantity < 50` (or a specified threshold) and returns the list to the GUI for display.

B. Edge Case Handling

- **Insufficient Stock:** When a customer tries to purchase more than available, the transaction logic detects this, executes a ROLLBACK, and returns `False`, ensuring the database state is never corrupted.
- **Invalid Login:** The `verify_login` function returns `None` for invalid credentials, preventing access.
- **Database Errors:** All `DBManager` methods include robust `try/except` blocks that execute a ROLLBACK on error, preventing partial transactions and printing the detailed error message to the console.

C. Unit Test Summary

- **Requirement:** Unit tests were required to verify correct functionality.

- **Implementation:** A dedicated Python test file (unittests.py) was implemented using the unittest framework.
- **Coverage:** The tests cover login, admin features (restock alerts), and the critical customer transaction process, including successful order placement and guaranteed rollback when stock is insufficient. The tests were run successfully, confirming the stability and correctness of the application logic.